

HTB ACADEMY

Introduction to Malware Analysis

HTB Academy · Malware Analysis Track · Study Notes

A hands-on walkthrough of static and dynamic malware analysis — PE inspection with pestudio, disassembly with x64dbg and IDA, hashing/imphashing, and dynamic behavioural analysis with Noriben. Lab answers are partially masked as a hint.

Platform	HackTheBox Academy
Module	Introduction to Malware Analysis
Progress	Static & dynamic analysis sections completed
Analyst	Chayan Panchal · @tron874
Core Skills	pestudio, x64dbg, IDA Pro, REMnux, Noriben, PE/hash analysis

0 Quick Reference — Read This First



Malware analysis — tool quick index

```
pestudio sample.exe                # static PE triage, no execution
Get-FileHash -Algorithm SHA256 <file>  # PowerShell file hashing
python3 imphash_calc.py <file>        # REMnux import-hash calculation
x64dbg / IDA Pro                    # disassembly & static code analysis
Noriben (wraps Procmon)             # dynamic/behavioural sandbox report
```

1 Static Analysis — PE Structure with pestudio

pestudio inspects a Windows executable's structure without ever running it — headers, sections, imports, and indicators are all visible up front, which makes it the right first step before anything gets executed in a sandbox.



Triaging potato.exe with pestudio

1. LOAD THE SAMPLE

```
pestudio C:\Samples\MalwareAnalysis\potato.exe
```

2. CHECK INDICATORS

```
imports (flag), footprints (11) reviewed first for quick red flags
```

3. INSPECT SECTIONS

```
sections tab -> .text section -> entropy calculated
```



pestudio finding (masked)

```
potato.exe .text section entropy -> 5.85
```

NOTE Entropy is a quick packing/encryption tell — values approaching 8.0 (out of a theoretical max of 8) suggest packed or encrypted content; a mid-range value like this is closer to normal compiled code.

2 Static Analysis — Disassembly with x64dbg & IDA

Where pestudio gives structure, a disassembler shows what the code actually does at the instruction level. x64dbg is used here to browse an executable's exported symbols; IDA Pro is used for deeper static analysis of a second sample.

```
x64dbg — exported symbol lookup (masked)

x64dbg -> potato.exe -> Symbols tab -> kernelbase.dll exports
Exported Kernel32.dll-family function starting with 'Attach' -> A.....C.....e
```

A second sample (orange.exe, from an additional password-protected sample set) was disassembled in IDA Pro. Decoded strings inside it reference a self-identifying label ("Gremlin") and registry API calls (RegOpenKeyExA, lstrcpA) consistent with a persistence mechanism being set up.

```
IDA Pro — persistence mechanism (masked)

IDA -> orange.exe -> decoded strings reveal registry API calls
RegOpenKeyExA(..., "Soft...ware...", ...) ; builds a registry path at runtime
Registry key modified for persistence -> S.....d.....n
```

MITRE A registry Run key is one of the most common and simplest Windows persistence techniques (MITRE T1547.001) — anything placed there executes automatically at every user logon.

3 Hashing & Imphashing

Cryptographic hashes uniquely fingerprint a specific file; the import hash (imphash) instead fingerprints a binary's *import table order*, which stays stable across recompiles and is often shared across samples from the same malware family or builder — making it useful for pivoting to related samples even when the file hash doesn't match.



PowerShell — SHA256 hashing

```
PS C:\> Get-FileHash -Algorithm SHA256 C:\Samples\MalwareAnalysis\dharma_sample.exe

SHA256 hash -> B.....B.....1
```



REMnux — import hash calculation

```
# On REMnux (a Linux distro purpose-built for malware analysis)
$ python3 imphash_calc.py /home/htb-student/Samples/MalwareAnalysis/potato.exe

IMPHASH -> 3.....0.....9
```

4 Dynamic Analysis with Noriben

Static analysis has limits — some behaviour only shows up once code actually runs. Noriben wraps Sysinternals' Process Monitor to record everything a sample does at runtime (process creation, file activity, network calls) into a single readable report, without requiring a full custom sandbox setup.



Dynamic Analysis Workflow — shell.exe

1. START NORIBEN

Begin capture before executing the sample

2. RUN THE SAMPLE

Execute shell.exe; Noriben records the resulting process tree

3. REVIEW THE REPORT

CreateProcess / File Activity sections show what actually ran



Noriben report excerpt (masked)

```
[CreateProcess] shell.exe:6640 > "%WinDir%\System32\cmd.exe /k ping <target> -n 5"
IP address shell.exe pings -> 1•••0•••1
```

NOTE The process tree (svchost -> explorer -> shell.exe -> cmd.exe -> ping) is exactly the kind of parent-child chain a SIEM/EDR detection rule would be built to flag — an unexpected child of explorer.exe spawning a shell is inherently suspicious regardless of what it pings.